

NPTEL R Programming – Week 6 Practical Record

Prepared from the uploaded NPTEL Foundations of R Software notes. The notes cover Lectures 22–25: loops, functions, and sequences.

```
# =====
# NPTEL FOUNDATIONS OF R SOFTWARE - WEEK 6 PRACTICAL
# Topics from the uploaded notes (Lectures 22-25):
# for loop, nested for loop, break, next, while, repeat,
# functions, and sequences using seq()
# =====

cat("\n===== NPTEL R PROGRAMMING WEEK 6 PRACTICAL =====\n")

# =====
# 1. FOR LOOP
# =====

cat("\n--- 1. FOR LOOP: Example 1 ---\n")
for (i in 1:5) {
  print(i^2)
}

cat("\n--- 2. FOR LOOP: Example 2 ---\n")
for (i in c(2, 4, 6, 7)) {
  print(i^2)
}

cat("\n--- 3. FOR LOOP WITH A FUNCTION ---\n")
x <- c(2, 4, 6, 8, 10, 12)

excount <- function(x) {
  count <- 0
  for (xval in x) {
    if (xval / 2 > 3)
      count <- count + 1
  }
  print(count)
}

excount(x)

# =====
# 2. NESTED FOR LOOP
# =====

cat("\n--- 4. NESTED FOR LOOP ---\n")
child <- c("child1", "child2", "child3")
sweet <- c("sweet1", "sweet2", "sweet3")

for (x in child) {
  for (y in sweet) {
    print(paste(x, y))
  }
}

# =====
# 3. BREAK COMMAND
# =====

cat("\n--- 5. BREAK COMMAND ---\n")
drink <- c("coffee", "lemonade", "tea", "juice")

for (x in drink) {
  if (x == "tea") {
    break
  }
  print(x)
}

# =====
# 4. NEXT COMMAND
# =====

cat("\n--- 6. NEXT COMMAND: Skip lemonade ---\n")
drink <- c("coffee", "lemonade", "tea", "juice")

for (x in drink) {
  if (x == "lemonade") {
    next
  }
  print(x)
}

cat("\n--- 7. NEXT COMMAND: Skip tea ---\n")
drink <- c("coffee", "lemonade", "tea", "juice")
```

```

for (x in drink) {
  if (x == "tea") {
    next
  }
  print(x)
}

# =====
# 5. WHILE LOOP
# =====

cat("\n--- 8. WHILE LOOP: Example 1 ---\n")
i <- 1

while (i < 10) {
  print(i^2)
  i <- i + 2
}

cat("\n--- 9. WHILE LOOP: Example 2 ---\n")
# Interactive example from the notes:
sumfunction <- function() {
  sum_value <- 0
  number <- as.integer(
    readline(prompt = "Please select any number less than 25: ")
  )

  while (number <= 25) {
    sum_value <- sum_value + number
    number <- number + 1
  }

  print(paste(
    "The sum of numbers received from the While Loop:",
    sum_value
  ))
}

# Run the interactive example when needed:
# sumfunction()

# =====
# 6. REPEAT LOOP
# =====

cat("\n--- 10. REPEAT LOOP: Example 3 ---\n")
i <- 1

repeat {
  print(i^2)
  i <- i + 2

  if (i > 10)
    break
}

cat("\n--- 11. REPEAT LOOP WITH NEXT AND BREAK ---\n")
i <- 1

repeat {
  i <- i + 1

  if (i < 10)
    next

  print(i^2)

  if (i >= 13)
    break
}

# =====
# 7. FUNCTIONS
# =====

cat("\n--- 12. FUNCTION: Single Variable ---\n")
abc <- function(x) {
  x^2
}

print(abc(3))
print(abc(6))
print(abc(9))

cat("\n--- 13. FUNCTION: Two Variables ---\n")
abc <- function(x, y) {
  x^2 + y^2
}

print(abc(3, 4))

```

```

print(abc(10, 10))
print(abc(-2, -3))

cat("\n--- 14. FUNCTION: sin(x)^2 + cos(x)^2 + x ---\n")
abc <- function(x) {
  sin(x)^2 + cos(x)^2 + x
}

print(abc(9))
print(abc(99))
print(abc(-15))

cat("\n--- 15. FUNCTION WITHOUT ARGUMENT ---\n")
abc <- function() {
  for (i in 1:3) {
    print(i^3)
  }
}

abc()

# =====
# 8. SEQUENCES USING seq()
# =====

cat("\n--- 16. SEQUENCE: Default Increment +1 ---\n")
print(seq(from = 2, to = 4))
print(seq(from = 4, to = 2))
print(seq(from = -4, to = 4))

cat("\n--- 17. SEQUENCE: Constant Increment +2 ---\n")
print(seq(from = 10, to = 20, by = 2))

cat("\n--- 18. SEQUENCE: Constant Decrement -2 ---\n")
print(seq(from = 20, to = 10, by = -2))

cat("\n--- 19. SEQUENCE: Fractional Decrement -0.5 ---\n")
print(seq(from = 3, to = -2, by = -0.5))

cat("\n--- 20. SEQUENCE: Predefined Length ---\n")
print(seq(to = 10, length.out = 10))
print(seq(from = 10, length.out = 10))

cat("\n--- 21. SEQUENCE: Fractional Increment +0.1 ---\n")
print(seq(from = 10, length.out = 10, by = 0.1))

cat("\n--- 22. SEQUENCE: Predefined Length with Decrement -2 ---\n")
print(seq(from = 10, length.out = 10, by = -2))

cat("\n--- 23. SEQUENCE: Fractional Decrement -0.2 ---\n")
print(seq(from = 10, length.out = 5, by = -0.2))

cat("\n===== PRACTICAL COMPLETED =====\n")

```